



CURSO TECNICAS DE PROGRAMACION ORIENTADA A OBJETOS

DESARROLLO T1

“INFORME DESARROLLO DE CASOS”

Docente:

- MARTIN EDUARDO TORRES RODRIGUEZ.

Estudiantes:

- García Bazán Carlos Alfredo (N00403695)
- Barrera Neyra Bruce Williams (N00439177)
- Hurtado Mendez Arturo Miguel (N0043882)
- Trigos Chuan Rony (N00427870)
- Silvia Oria Paulino Ismael (N00403695)

Número de grupo: 5

LIMA

2026

I. INTRODUCCIÓN

La programación orientada a objetos en Java constituye una metodología fundamental para el desarrollo de aplicaciones modernas, permitiendo estructurar el código de manera eficiente y reutilizable. A lo largo de los casos que se presentan en este documento, se abordan conceptos clave como el encapsulamiento, el uso de constructores, la definición de métodos y atributos estáticos, el manejo de errores y excepciones, así como la representación de clases y relaciones mediante diagramas UML. Además, se profundiza en los principios de herencia y polimorfismo, esenciales para crear sistemas flexibles y escalables. Estos temas son aplicados en ejemplos prácticos, facilitando la comprensión y el dominio de las técnicas propias de la programación orientada a objetos en Java.

II. DESARROLLO

2. DESARROLLO Y ANALISIS DE CASOS

2.1. Caso 1: Fundamentos de POO

- a) Crear clase estudiante.

```
public class Estudiante {  
    // Atributos privados (encapsulamiento)  
    private String nombre;  
    private int edad;  
    private String carrera;
```

- b) Implementar constructor, métodos get/set y método para mostrar datos

```
    // Constructor  
    public Estudiante(String nombre, int edad, String carrera) {  
        this.nombre = nombre;  
        this.edad = edad;  
        this.carrera = carrera;  
    }  
    // Métodos GET  
    public String getNombre() {  
        return nombre;  
    }  
  
    public int getEdad() {  
        return edad;  
    }  
  
    public String getCarrera() {  
        return carrera;  
    }  
  
    // Métodos SET  
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }  
}
```

```

public void setEdad(int edad) {
    this.edad = edad;
}

public void setCarrera(String carrera) {
    this.carrera = carrera;
}

// Método para mostrar datos
public void mostrarDatos() {
    System.out.println("Nombre: " + nombre);
    System.out.println("Edad: " + edad);
    System.out.println("Carrera: " + carrera);
    System.out.println("-----");
}

```

- c) Registrar 3 estudiantes

```

// Registrar estudiantes
Estudiante est1 = new Estudiante("Rony", 29, "Ingeniería de
Sistemas");
Estudiante est2 = new Estudiante("Paul", 25, "Ingeniería de
Sistemas");
Estudiante est3 = new Estudiante("Bruce", 26, "Ingeniería de
Sistemas");

```

- d) Análisis: explicación de encapsulamiento y uso de constructores

Encapsulamiento

En la clase **Estudiante**, se aplica encapsulamiento cuando se hace esto:

```

private String nombre;
private int edad;
private String carrera;

```

Al usar **private**, se evita que otras clases modifiquen estos atributos directamente.

En lugar de eso, se usa los métodos **getters** y **setters**:

getNombre(), getEdad(), getCarrera() → permiten leer los valores.

setNombre(), setEdad(), setCarrera() → permiten modificar los valores de forma controlada.

Estos métodos nos permiten tanto leer los valores como modificarlos dentro del código, garantizando así que el acceso a los atributos sea seguro y controlado. De esta forma, se protege la integridad de los datos y se favorece la reutilización del código, siguiendo los principios de la programación orientada a objetos.

Uso del Constructor

El constructor es un método especial que se ejecuta automáticamente cuando se crea un objeto.

En el caso:

```
public Estudiante (String nombre, int edad, String carrera) {

    this.nombre = nombre;

    this.edad = edad;

    this.carrera = carrera;

}
```

Esto nos sirve para inicializar los atributos del objeto en el momento de su creación.

Cuando se hace:

```
Estudiante est1 = new Estudiante ("Rony", 29, "Ingeniería de
Sistemas");
```

Estamos:

- ✓ Creando un objeto.
- ✓ Asignando valores desde el inicio.
- ✓ Evitando tener que usar varios `set` después.

Esto facilita la legibilidad y el mantenimiento del código, simplificando así el proceso de instanciación y mejorando la robustez del programa.

2.2. CASO 2:

2.2.1 Código java del caso 2:

El código de este caso estará dividido 4 partes (1 package com.mycompany SistemaBancario) y 3 Java Class (SistemaBancario, SaldoInsuficienteException y CuentaBancaria.

Clase.

2.2.1.1 SistemaBancario(package com.mycompany.sistemabancario):

Código:

/*

* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license

*/

```
package com.mycompany.sistemabancario;
```

```
import java.util.Scanner;
```

```
import sistemabancario.CuentaBancaria;
```

```
public class SistemaBancario {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        CuentaBancaria cuenta = null;

        System.out.println("=== SISTEMA BANCARIO ===");
        System.out.print("Ingrese el saldo inicial para la nueva cuenta: $");

        double saldoInicial = scanner.nextDouble();

        try {
            if (saldoInicial < 0) {
                throw new IllegalArgumentException("El saldo inicial no puede ser negativo");
            }
            cuenta = new CuentaBancaria(saldoInicial);
        } catch (IllegalArgumentException e) {
            System.out.println("Error al crear cuenta: " + e.getMessage());
            System.exit(1);
        }

        // Menú de operaciones
        boolean continuar = true;
        while (continuar) {
            System.out.println("\n=== Menú de OPERACIONES ===");
            System.out.println("1. Depositar dinero");
            System.out.println("2. Retirar dinero");
            System.out.println("3. Consultar saldo");
            System.out.println("4. Ver información de la cuenta");
            System.out.println("5. Crear nueva cuenta");
            System.out.println("6. Ver total de cuentas");
            System.out.println("7. Salir");
            System.out.print("Seleccione una opción: ");

            int opcion = scanner.nextInt();

            switch (opcion) {
                case 1:
                    System.out.print("Ingrese monto a depositar: $");
                    double deposito = scanner.nextDouble();
                    cuenta.depositar(deposito);
                    break;

                case 2:
                    System.out.print("Ingrese monto a retirar: $");
                    double retiro = scanner.nextDouble();
```

```
        cuenta.retirar(retiro);
        break;

    case 3:
        System.out.println("Saldo Actual: $" + cuenta.getSaldo());
        break;

    case 4:
        cuenta.mostrarInformacion();
        break;

    case 5:
        System.out.print("Ingresar el saldo inicial para la nueva cuenta: $");
        saldoInicial = scanner.nextDouble();
        try {
            if (saldoInicial < 0) {
                throw new IllegalArgumentException("El saldo inicial no puede ser
negativo");
            }
            cuenta = new CuentaBancaria(saldoInicial);
        } catch (IllegalArgumentException e) {
            System.out.println("Error al crear cuenta: " + e.getMessage());
        }
        break;

    case 6:
        CuentaBancaria.mostrarTotalCuentas();
        break;

    case 7:
        continuar = false;
        System.out.println("¡Gracias por usar el sistema bancario!");
        break;

    default:
        System.out.println("Opción no válida");
    }
}
scanner.close();
}
```

2.2.1.2 Clase CuentaBancaria(java):

La clase CuentaBancaria es nuestra clase principal que nos servirá para representar una cuenta bancaria para luego definir su lógica(comportamiento).

Código:

```
package sistemabancario;

public class CuentaBancaria {

    // Atributos

    private double saldo;
    private static int contadorCuentas = 0;
    private final int numeroCuenta;

    // Constructor

    public CuentaBancaria(double saldoInicial) {
        this.saldo = saldoInicial;
        contadorCuentas++;
        this.numeroCuenta = contadorCuentas;
        System.out.println("¡Cuenta creada exitosamente! ");
        System.out.println("Número de cuenta: " + numeroCuenta);
        System.out.println("Su saldo inicial: $" + saldoInicial);
    }

    // Método para depositar

    public void depositar(double monto) {
        try {
            if (monto <= 0) {
                throw new IllegalArgumentException("El monto a depositar debe ser mayor que cero");
            }
            saldo += monto;
        }
    }
}
```

```
        System.out.println("Se realizó el depósito exitosamente: $" + monto);
        System.out.println("Nuevo saldo: $" + saldo);
    } catch (IllegalArgumentException e) {
        System.out.println("Error en depósito: " + e.getMessage());
    }
}

// Método para retirar
public void retirar(double monto) {
    try {
        if (monto <= 0) {
            throw new IllegalArgumentException("El monto a retirar debe ser mayor a cero");
        }
        if (monto > saldo) {
            throw new SaldoInsuficienteException("Saldo insuficiente. Saldo actual: $" + saldo);
        }
        saldo -= monto;
        System.out.println("Retiro exitoso: $" + monto);
        System.out.println("Nuevo saldo: $" + saldo);
    } catch (IllegalArgumentException | SaldoInsuficienteException e) {
        System.out.println("Error en retiro: " + e.getMessage());
    }
}

// Getters
public double getSaldo() {
    return saldo;
}
```

```
public int getNumeroCuenta() {  
    return numeroCuenta;  
}  
  
// Métodos estáticos  
public static int getTotalCuentas() {  
    return contadorCuentas;  
}  
  
public static void mostrarTotalCuentas() {  
    System.out.println("\n=== TOTAL DE CUENTAS CREADAS: " +  
contadorCuentas + " ===");  
}  
  
// Mostrar información de la cuenta  
public void mostrarInformacion() {  
    System.out.println("\n=== INFORMACIÓN DE LA CUENTA ===");  
    System.out.println("Número de cuenta: " + numeroCuenta);  
    System.out.println("Saldo actual: $" + saldo);  
    System.out.println("=====");  
}  
}
```

2.2.3.1 Clase SistemaBancario:

En esta Clase Main se usará para probar el funcionamiento logrando hacer las cualquiera de las 7 operaciones.

Código:

```
package sistemabancario;
```

```
import java.util.Scanner;
```

```
public class SistemaBancario {
```

```
    public static void main(String[] args) {
```

```
        Scanner scanner = new Scanner(System.in);
```

```
        CuentaBancaria cuenta = null;
```

```
        System.out.println("=== SISTEMA BANCARIO ===");
```

```
        System.out.print("Ingrese el saldo inicial para la nueva cuenta: $");
```

```
        double saldoInicial = scanner.nextDouble();
```

```
        try {
```

```
            if (saldoInicial < 0) {
```

```
                throw new IllegalArgumentException("El saldo inicial no puede ser negativo");
```

```
            }
```

```
            cuenta = new CuentaBancaria(saldoInicial);
```

```
        } catch (IllegalArgumentException e) {
```

```
            System.out.println("Error al crear cuenta: " + e.getMessage());
```

```
            System.exit(1);
```

```
        }
```

```
        // Menú de operaciones
```

```
        boolean continuar = true;
```

```
while (continuar) {  
    System.out.println("\n=== Menú de OPERACIONES ===");  
    System.out.println("1. Depositar dinero");  
    System.out.println("2. Retirar dinero");  
    System.out.println("3. Consultar saldo");  
    System.out.println("4. Ver información de la cuenta");  
    System.out.println("5. Crear nueva cuenta");  
    System.out.println("6. Ver total de cuentas");  
    System.out.println("7. Salir");  
    System.out.print("Seleccione una opción: ");  
  
    int opcion = scanner.nextInt();  
  
    switch (opcion) {  
        case 1:  
            System.out.print("Ingrese monto a depositar: $");  
            double deposito = scanner.nextDouble();  
            cuenta.depositar(deposito);  
            break;  
  
        case 2:  
            System.out.print("Ingrese monto a retirar: $");  
            double retiro = scanner.nextDouble();  
            cuenta.retirar(retiro);  
            break;  
  
        case 3:  
            System.out.println("Saldo Actual: $" + cuenta.getSaldo());  
            break;
```

case 4:

```
cuenta.mostrarInformacion();  
break;
```

case 5:

```
System.out.print("Ingresar el saldo inicial para la nueva cuenta: $");  
saldoInicial = scanner.nextDouble();  
try {  
    if (saldoInicial < 0) {  
        throw new IllegalArgumentException("El saldo inicial no puede  
ser negativo");  
    }  
    cuenta = new CuentaBancaria(saldoInicial);  
} catch (IllegalArgumentException e) {  
    System.out.println("Error al crear cuenta: " + e.getMessage());  
}  
break;
```

case 6:

```
CuentaBancaria.mostrarTotalCuentas();  
break;
```

case 7:

```
continuar = false;  
System.out.println("¡Gracias por usar el sistema bancario!");  
break;
```

default:

```
System.out.println("Opción no válida");  
}  
}
```

```
        scanner.close();  
    }  
}
```

2.2.3.1 Análisis del Caso 2:

En el caso 2 usaremos 4 elementos del mecanismo de control (encapsulación, static , validaciones y excepciones con la finalidad de lograr los resultados propuestos en la T1.

Estático - Distribución de Recursos :

El variable estática contadorCuentas es de la clase no de instancias. Toda cuenta bancaria lo comparte y modifica cuantos cuentas se an creado en todo el sistema.

Estático - Acceso Directo :

El metodo estático getTotalCuentas podria ser llamado directo usando el nombre de la clase (CuentaBancaria getTotalCuentas) sin necesitar tener una instancia en particular de una cuenta. Esto tiene sentido porque la informacion es global para todo el sistema.

Encapsulamiento - Proteccion de Datos:

El atributo saldo se declara como private. Esto protege el dato interno, impidiendo que se modifique directo desde afuera de la clase. El acceso solo es permitido por medio de metodos publicos como getSaldo, depositar, y retirar, los cuales pueden incluir logica de validacion (como previniendo saldos negativos).

Herencia - Excepciones Propias :

La excepcion personalizada (ejemplo SaldoInsuficienteException) se deriva de la clase base Exception. Esto permite creaciones tipos de errores especificos del dominio del problema (un sistema bancario) que pueden ser capturados y administrados selectivamente. Polimorfismo - Manejo De Errores

La sección try-catch es capaz de atrapar un error de forma genérica (Exception) o atrapar varios errores específicos (IllegalArgumentException, SaldoInsuficienteException) utilizando idéntica sintaxis. El polimorfismo hace posible gestionar clases de error distintas de manera uniforme a través de su ruta común (la clase Exception).

Abstracción - Ocultando El Complejidad

El usuario del sistema observa una interfaz sencilla con opciones como "Depositar dinero" o "Consultar saldo". La abstracción oculta la complejidad interna de cómo se ejecutan esas operaciones (las validaciones, los cálculos, las actualizaciones en la base de datos, etc.), mostrando únicamente lo esencial para interactuar con el objeto.

Las funcionalidades del Caso dos son:

- Creación y Conteo de cuentas con un saldo inicial
- Validación de Depósitos, Retiros dentro de las cuentas bancarias
- Manejo Robusto de errores (Usando la clase `SaldoInsuficienteException` para restringir o aislar escenarios imposibles en el mundo real).

2.3. CASO 3

// Clase Categoria

```
package com.mycompany.caso3;
```

```
class Categoria {
```

```
    private int    idCategoria;  
    private String nombre;  
    private String descripcion;
```

```
    public Categoria(int idCategoria, String nombre, String descripcion) {  
        this.idCategoria = idCategoria;  
        this.nombre      = nombre;  
        this.descripcion = descripcion;  
    }  
    // — Getters y Setters
```

```
    public int getIdCategoria() {  
        return idCategoria;  
    }
```

```
    public String getNombre() {  
        return nombre;  
    }
```

```
    public void setNombre(String nombre) {  
        if (nombre == null || nombre.trim().isEmpty()) {
```

```

        System.out.println("[ERROR] El nombre de la categoría no puede estar
vacío.");
        return;
    }
    this.nombre = nombre;
}

public String getDescripcion() {
    return descripcion;
}

public void setDescripcion(String descripcion) {
    this.descripcion = descripcion;
}

// — Método de representación

```

```

    @Override
    public String toString() {
        return "Categoría{id=" + idCategoria + ", nombre=" + nombre + ",
descripcion=" + descripcion + "}";
    }
}

```

// Clase Inventario

```

package com.mycompany.caso3;
import java.util.ArrayList;

public class Inventario {

    private int    idInventario;
    private String fecha;
    private int    totalProductos;

    // Agregación: la lista de productos existe independientemente del inventario
    private ArrayList<Producto> productos;

    // Composición: los movimientos pertenecen al inventario
    private ArrayList<MovimientoStock> movimientos;

    public Inventario(int idInventario, String fecha) {
        this.idInventario = idInventario;
        this.fecha        = fecha;
    }
}

```

```

        this.totalProductos = 0;
        this.productos    = new ArrayList<>();
        this.movimientos  = new ArrayList<>();
    }

    public int getIdInventario() { return idInventario; }
    public String getFecha()     { return fecha; }
    public int getTotalProductos() { return totalProductos; }

    // RF-01: Registrar Producto

    /**
     * Agrega un producto al inventario. Valida que no exista un ID duplicado
     * y que los campos obligatorios sean correctos (HU-01, Criterio
    exitoso/fallido).
     */
    public void agregarProducto(Producto producto) {
        if (producto == null) {
            System.out.println("[ERROR] El producto no puede ser nulo.");
            return;
        }
        // Validación: nombre vacío o precio negativo
        if (producto.getId() == null || producto.getId().trim().isEmpty())
        {
            System.out.println("[ERROR] El nombre del producto es obligatorio. No
    se registró.");
            return;
        }
        if (producto.getPrecio() < 0) {
            System.out.println("[ERROR] El precio es negativo. No se registró.");
            return;
        }
        // Validación: ID duplicado
        if (buscarProductoPorId(producto.getId()) != null) {
            System.out.println("[ERROR] Ya existe un producto con ID " +
    producto.getId() + ".");
            return;
        }
        productos.add(producto);
        totalProductos = productos.size();
        System.out.println("[OK] Producto '" + producto.getId() + "'
    registrado correctamente.");
    }

```

```
// RF-04: Eliminar Producto
```

```
/**
 * Elimina un producto por su ID. No permite eliminar si tiene movimientos
 activos.
 */
public void eliminarProducto(int id) {
    Producto p = buscarProductoPorId(id);
    if (p == null) {
        System.out.println("[ERROR] No se encontró un producto con ID " + id
+ ".");
        return;
    }
    // Verificar si tiene movimientos activos
    for (MovimientoStock mov : movimientos) {
        if (mov.getProductoAsociado() != null &&
            mov.getProductoAsociado().getId() == id) {
            System.out.println("[ERROR] El producto '" + p.getNombre() +
                "' tiene movimientos activos y no puede eliminarse.");
            return;
        }
    }
    productos.remove(p);
    totalProductos = productos.size();
    System.out.println("[OK] Producto '" + p.getNombre() + "' eliminado del
inventario.");
}
```

```
// RF-03: Buscar Producto
```

```
/** Busca por nombre o categoría (búsqueda parcial, insensible a
mayúsculas). */
public Producto buscarProducto(String criterio) {
    if (criterio == null || criterio.trim().isEmpty()) {
        System.out.println("[ERROR] El criterio de búsqueda no puede estar
vacío.");
        return null;
    }
    String criterioMin = criterio.toLowerCase();
    for (Producto p : productos) {
        if (p.getNombre().toLowerCase().contains(criterioMin) ||
            (p.getCategoria() != null &&
             p.getCategoria().toLowerCase().contains(criterioMin)))
        {
            return p;
        }
    }
}
```

```

    }
  }
  System.out.println("[INFO] No se encontró ningún producto con el criterio:
  "" + criterio + " ");
  return null;
}

/** Busca por ID exacto. */
public Producto buscarProductoPorId(int id) {
  for (Producto p : productos) {
    if (p.getId() == id) return p;
  }
  return null;
}

/** Lista todos los productos que coinciden con el criterio. */
public ArrayList<Producto> buscarTodos(String criterio) {
  ArrayList<Producto> resultado = new ArrayList<>();
  String criterioMin = criterio.toLowerCase();
  for (Producto p : productos) {
    if (p.getNombre().toLowerCase().contains(criterioMin) ||
        (p.getCategoria() != null &&
         p.getCategoria().getNombre().toLowerCase().contains(criterioMin)))
    {
      resultado.add(p);
    }
  }
  return resultado;
}

// RF-02: Actualizar Stock (registrar movimiento)

/**
 * Registra un movimiento de stock (entrada o salida) y lo guarda en el
inventario.
 * Aplica la composición: el movimiento pertenece al inventario.
 */
public void registrarMovimiento(MovimientoStock movimiento) {
  if (movimiento == null) {
    System.out.println("[ERROR] El movimiento no puede ser nulo.");
    return;
  }
  boolean exito = movimiento.registrarMovimiento();
  if (exito) {
    movimientos.add(movimiento);
  }
}

```

```

    }

    // RF-05: Generar Reporte

    /**
     * Genera y devuelve un reporte completo del inventario:
     * productos, stock y valor total (HU-03).
     */
    public String generarReporte() {
        StringBuilder sb = new StringBuilder();
        String linea = "=".repeat(70);

        sb.append(linea).append("\n");
        sb.append("    REPORTE DE INVENTARIO\n");
        sb.append("    Fecha del sistema: ").append(fecha).append("\n");
        sb.append(linea).append("\n");

        if (productos.isEmpty()) {
            // Criterio fallido HU-03: sin productos registrados
            sb.append(" [INFO] No hay productos registrados en el inventario.\n");
            sb.append(linea).append("\n");
            return sb.toString();
        }

        // Encabezado de tabla
        sb.append(String.format(" %-4s %-22s %-14s %10s %8s %14s\n",
            "ID", "PRODUCTO", "CATEGORÍA", "PRECIO", "STOCK",
            "VALOR TOTAL"));
        sb.append("-".repeat(70)).append("\n");

        double valorTotalInventario = 0;
        for (Producto p : productos) {
            double valorProducto = p.getPrecio() * p.getStock();
            valorTotalInventario += valorProducto;
            String catNombre = p.getCategoria() != null ?
                p.getCategoria().getNombre() : "Sin cat.";
            sb.append(String.format(" %-4d %-22s %-14s %10.2f %8d %14.2f\n",
                p.getId(), p.getNombre(), catNombre,
                p.getPrecio(), p.getStock(), valorProducto));
        }

        sb.append("-".repeat(70)).append("\n");
        sb.append(String.format(" Total de productos: %d\n", totalProductos));
        sb.append(String.format(" Valor total del inventario: S/. %.2f\n",
            valorTotalInventario));
        sb.append(linea).append("\n");
    }

```

```

// Resumen de movimientos
if (!movimientos.isEmpty()) {
    sb.append("\n HISTORIAL DE MOVIMIENTOS DE STOCK\n");
    sb.append("-".repeat(70)).append("\n");
    for (MovimientoStock mov : movimientos) {
        sb.append(" ").append(mov.toString()).append("\n");
    }
    sb.append(linea).append("\n");
}

return sb.toString();
}

@Override
public String toString() {
    return "Inventario{id=" + idInventario + ", fecha=" + fecha +
        ", totalProductos=" + totalProductos + "}";
}
}

```

// Clase Inventario Main

```

package com.mycompany.caso3;
public class Main {

    public static void main(String[] args) {

        System.out.println("=====
        =====");
        System.out.println("|| SISTEMA DE GESTIÓN DE INVENTARIO –
        Caso 3 UML ||");

        System.out.println("=====
        =====\n");

        // 1. CREAR CATEGORÍAS (Composición con Producto)

        System.out.println("Creando categorías...");
        Categoria catElectronica = new Categoria(1, "Electrónica", "Dispositivos
        y accesorios electrónicos");
        Categoria catAlimentos = new Categoria(2, "Alimentos", "Productos de
        consumo alimenticio");
        Categoria catLimpieza = new Categoria(3, "Limpieza", "Artículos de
        higiene y limpieza");
    }
}

```

```
System.out.println(" " + catElectronica);
System.out.println(" " + catAlimentos);
System.out.println(" " + catLimpieza);
```

// 2. CREAR PROVEEDORES (Asociación con Producto)

```
System.out.println("\nCreando proveedores...");
Proveedor prov1 = new Proveedor(1, "TechDistrib SAC", "987654321",
"ventas@techdistrib.pe");
Proveedor prov2 = new Proveedor(2, "AlimPeru EIRL", "912345678",
"pedidos@alimperu.pe");
Proveedor prov3 = new Proveedor(3, "LimpiaMax S.A.", "933221100",
"contacto@limpiamax.pe");
System.out.println(" " + prov1);
System.out.println(" " + prov2);
System.out.println(" " + prov3);
```

// 3. CREAR PRODUCTOS (RF-01 – Registrar Producto | HU-01)

```
System.out.println("\nCreando productos (RF-01)...");
Producto p1 = new Producto(101, "Laptop HP 15\"", "Laptop Intel Core
i5, 8GB RAM, 512GB SSD", 3500.00, 15, catElectronica);
Producto p2 = new Producto(102, "Mouse Inalámbrico", "Mouse USB
2.4GHz, 1600 DPI", 45.00, 80, catElectronica);
Producto p3 = new Producto(103, "Arroz Extra", "Bolsa de arroz de 5
kg, grano largo", 18.50, 200, catAlimentos);
Producto p4 = new Producto(104, "Aceite Vegetal", "Botella de aceite
vegetal 1 litro", 9.90, 150, catAlimentos);
Producto p5 = new Producto(105, "Detergente Ariel", "Detergente en polvo
2 kg, fragancia fresh", 22.00, 60, catLimpieza);
```

// Agregar proveedores a productos (Asociación)

```
p1.agregarProveedor(prov1);
p2.agregarProveedor(prov1);
p3.agregarProveedor(prov2);
p4.agregarProveedor(prov2);
p5.agregarProveedor(prov3);
p1.agregarProveedor(prov3); // un producto puede tener varios proveedores
```

// 4. CREAR INVENTARIO y AGREGAR PRODUCTOS (Agregación)

```
System.out.println("\nCreando inventario y registrando productos...");
Inventario inventario = new Inventario(1, "2026-04-25");
inventario.agregarProducto(p1);
```

```
inventario.agregarProducto(p2);
inventario.agregarProducto(p3);
inventario.agregarProducto(p4);
inventario.agregarProducto(p5);
```

```
// 5. VALIDACIONES – HU-01 Criterio Fallido (campos inválidos)
```

```
System.out.println("\n Probando validaciones (HU-01 – Criterio
fallido)...");
// Producto con nombre vacío
Producto invalido1 = new Producto(200, "", "sin nombre", 10.0, 5,
catLimpieza);
inventario.agregarProducto(invalido1);
// Producto con precio negativo
Producto invalido2 = new Producto(201, "Producto malo", "precio
negativo", -5.0, 5, catAlimentos);
inventario.agregarProducto(invalido2);
// ID duplicado
Producto duplicado = new Producto(101, "Copia Laptop", "duplicado",
100.0, 1, catElectronica);
inventario.agregarProducto(duplicado);
```

```
// 6. MOSTRAR DATOS DE UN PRODUCTO
```

```
System.out.println("\nDetalle de un producto:");
System.out.println(p1.mostrarDatos());
```

```
// 7. RF-03 – BUSCAR PRODUCTO
```

```
System.out.println("\nBúsqueda de producto (RF-03)...");
Producto encontrado = inventario.buscarProducto("Mouse");
if (encontrado != null) {
    System.out.println(" Resultado: " + encontrado);
}

Producto porCategoria = inventario.buscarProducto("Alimentos");
if (porCategoria != null) {
    System.out.println(" Por categoría: " + porCategoria);
}

// Búsqueda sin resultados
inventario.buscarProducto("Nevera");
```

```
// 8. RF-02 – ACTUALIZAR STOCK (movimientos de entrada y salida)
```

```
// HU-02: Criterio exitoso y fallido
```

```
System.out.println("\nRegistrando movimientos de stock (RF-02 |  
HU-02)...");
```

```
// ENTRADA exitosa
```

```
MovimientoStock mov1 = new
```

```
MovimientoStock(MovimientoStock.ENTRADA, 30, "2026-04-25", p2);
```

```
inventario.registrarMovimiento(mov1);
```

```
// SALIDA exitosa
```

```
MovimientoStock mov2 = new
```

```
MovimientoStock(MovimientoStock.SALIDA, 10, "2026-04-25", p3);
```

```
inventario.registrarMovimiento(mov2);
```

```
// SALIDA fallida (stock insuficiente – HU-02 Criterio fallido)
```

```
System.out.println("\n [Criterio Fallido HU-02]: Intentar sacar más stock  
del disponible:");
```

```
MovimientoStock mov3 = new
```

```
MovimientoStock(MovimientoStock.SALIDA, 999, "2026-04-25", p5);
```

```
inventario.registrarMovimiento(mov3);
```

```
// Otra entrada válida
```

```
MovimientoStock mov4 = new
```

```
MovimientoStock(MovimientoStock.ENTRADA, 50, "2026-04-25", p4);
```

```
inventario.registrarMovimiento(mov4);
```

```
// 9. RF-04 – ELIMINAR PRODUCTO
```

```
System.out.println("\nEliminando producto (RF-04)...");
```

```
// Intento fallido: producto con movimientos activos
```

```
System.out.println(" [Intentando eliminar producto con movimientos  
activos]:");
```

```
inventario.eliminarProducto(102); // p2 tiene mov1
```

```
// Eliminación exitosa: producto sin movimientos
```

```
Producto pTemp = new Producto(999, "Producto Temporal", "Solo para  
prueba", 5.0, 3, catLimpieza);
```

```
inventario.agregarProducto(pTemp);
```

```
inventario.eliminarProducto(999);
```

```
// 10. RF-05 – GENERAR REPORTE FINAL (HU-03)
```

```
System.out.println("\nGenerando reporte del inventario (RF-05 | HU-03)...\n");
System.out.println(inventario.generarReporte());
```

// 11. PROBAR SETTER CON VALIDACIÓN (Semana 2: manejo de errores)

```
System.out.println("Probando setters con validación (Semana 2 – manejo  
de errores)...");
```

```
prov1.setEmail("emailinvalido"); // sin @
prov1.setEmail("nuevo@correo.pe");
System.out.println(" Email actualizado: " + prov1.getEmail());
```

```
catElectronica.setNombre(""); // nombre vacío
catElectronica.setNombre("Tecnología");
System.out.println(" Categoría renombrada: " +
catElectronica.getNombre());
```

```
System.out.println("\n ┌───────────────────────────────────┐");  
    System.out.println(" │ Ejecución completada – Sistema de Inventario OK │");  
    System.out.println(" └───────────────────────────────────┘");  
  
}
```

```
// Clase MovimientoStock
```

```
package com.mycompany.caso3;
public class MovimientoStock {

    // — Constantes de tipo
    public static final String ENTRADA = "ENTRADA";
    public static final String SALIDA = "SALIDA";

    // — Contador estático
    private static int contadorId = 1;

    // — Atributos privados
```

```
private int    idMovimiento;
private String tipo;    // "ENTRADA" o "SALIDA"
private int    cantidad;
private String fecha;

// Dependencia
private Producto productoAsociado;

// Constructor
public MovimientoStock(String tipo, int cantidad, String fecha, Producto
productoAsociado) {
    this.idMovimiento = contadorId++;
    this.productoAsociado = productoAsociado;
    setTipo(tipo);
    setCantidad(cantidad);
    this.fecha = fecha;
}

// Getters
public int getIdMovimiento() {
    return idMovimiento;
}

public String getTipo() {
    return tipo;
}

public int getCantidad() {
    return cantidad;
}

public String getFecha() {
    return fecha;
}

public Producto getProductoAsociado() {
    return productoAsociado;
}

// Setters con validación
public void setTipo(String tipo) {
    if (ENTRADA.equalsIgnoreCase(tipo) || SALIDA.equalsIgnoreCase(tipo))
    {
        this.tipo = tipo.toUpperCase();
    } else {
        System.out.println("[ERROR] Tipo de movimiento inválido: " + tipo +
            ". Debe ser ENTRADA o SALIDA.");
    }
}
```

```

        this.tipo = ENTRADA; // valor por defecto
    }
}

public void setCantidad(int cantidad) {
    if (cantidad <= 0) {
        System.out.println("[ERROR] La cantidad debe ser mayor a cero.");
        this.cantidad = 1;
    } else {
        this.cantidad = cantidad;
    }
}

public boolean registrarMovimiento() {
    if (productoAsociado == null) {
        System.out.println("[ERROR] No hay producto asociado al
movimiento.");
        return false;
    }

    if (tipo.equals(ENTRADA)) {
        productoAsociado.setStock(productoAsociado.getStock() + cantidad);
        System.out.println("[OK] ENTRADA registrada: +" + cantidad +
            " unidades de " + productoAsociado.getNombre() + ".");
        System.out.println("    Stock actualizado: " +
productoAsociado.getStock());
        return true;
    } else { // SALIDA
        if (cantidad > productoAsociado.getStock()) {
            System.out.println("[ERROR] Stock insuficiente para " +
                productoAsociado.getNombre() + ". Disponible: " +
                productoAsociado.getStock() + ", solicitado: " + cantidad);
            return false;
        }
        productoAsociado.setStock(productoAsociado.getStock() - cantidad);
        System.out.println("[OK] SALIDA registrada: -" + cantidad +
            " unidades de " + productoAsociado.getNombre() + ".");
        System.out.println("    Stock actualizado: " +
productoAsociado.getStock());
        return true;
    }
}

@Override
public String toString() {

```

```

        String prod = productoAsociado != null ? productoAsociado.getNombre() :
"N/A";
        return String.format("[Mov.%03d] %-8s | Producto: %-20s | Cant: %4d |
Fecha: %s",
            idMovimiento, tipo, prod, cantidad, fecha);
    }
}

```

// Clase Producto

```

package com.mycompany.caso3;
import java.util.ArrayList;

public class Producto {

    // — Atributos privados (encapsulamiento)
    private int    id;
    private String nombre;
    private String descripcion;
    private double precio;
    private int    stock;

    // Composición: Categoría es parte esencial del Producto
    private Categoría categoria;

    // Asociación: un Producto puede tener varios Proveedores (Semana 2 –
    Colecciones)
    private ArrayList<Proveedor> proveedores;

    // — Constructor
    public Producto(int id, String nombre, String descripcion,
        double precio, int stock, Categoría categoria) {
        this.id      = id;
        this.nombre   = nombre;
        this.descripcion = descripcion;
        setPrecio(precio);    // usa setter para validar
        setStock(stock);      // usa setter para validar
        this.categoria = categoria;
        this.proveedores = new ArrayList<>();
    }

    // — Getters
    public int getId() {
        return id;
    }

    public String getNombre() {

```

```
        return nombre;
    }

    public String getDescripcion() {
        return descripcion;
    }

    public double getPrecio() {
        return precio;
    }

    public int getStock() {
        return stock;
    }

    public Categoria getCategoria() {
        return categoria;
    }

    public ArrayList<Proveedor> getProveedores() {
        return proveedores;
    }

    // — Setters con validación
    public void setNombre(String nombre) {
        if (nombre == null || nombre.trim().isEmpty()) {
            System.out.println("[ERROR] El nombre del producto no puede estar
vacío.");
            return;
        }
        this.nombre = nombre;
    }

    public void setDescripcion(String descripcion) {
        this.descripcion = descripcion;
    }

    public void setPrecio(double precio) {
        if (precio < 0) {
            System.out.println("[ERROR] El precio no puede ser negativo. Se asigna
0.0.");
            this.precio = 0.0;
        } else {
            this.precio = precio;
        }
    }
}
```

```

    public void setStock(int stock) {
        if (stock < 0) {
            System.out.println("[ERROR] El stock no puede ser negativo. Se asigna
0.");
            this.stock = 0;
        } else {
            this.stock = stock;
        }
    }

    public void setCategoria(Categoria categoria) {
        this.categoria = categoria;
    }

    // — Métodos de negocio

    public void agregarProveedor(Proveedor proveedor) {
        if (proveedor != null) {
            proveedores.add(proveedor);
            System.out.println("[OK] Proveedor " + proveedor.getNombre() +
                " agregado al producto " + nombre + ".");
        }
    }

    /**
     * Muestra todos los datos del producto con formato legible (RF-05).
     */
    public String mostrarDatos() {
        StringBuilder sb = new StringBuilder();
        sb.append("=".repeat(50)).append("\n");
        sb.append(" PRODUCTO\n");
        sb.append("=".repeat(50)).append("\n");
        sb.append(" ID      : ").append(id).append("\n");
        sb.append(" Nombre  : ").append(nombre).append("\n");
        sb.append(" Descripción : ").append(descripcion).append("\n");
        sb.append(" Precio   : S/. ").append(String.format("%.2f",
precio)).append("\n");
        sb.append(" Stock    : ").append(stock).append(" unidades\n");
        sb.append(" Categoría : ").append(categoria != null ?
categoria.getNombre() : "Sin categoría").append("\n");
        if (!proveedores.isEmpty()) {
            sb.append(" Proveedores : ");
            for (int i = 0; i < proveedores.size(); i++) {
                if (i > 0) sb.append(", ");
                sb.append(proveedores.get(i).getNombre());
            }
        }
    }

```

```

        sb.append("\n");
    }
    sb.append("=".repeat(50));
    return sb.toString();
}

@Override
public String toString() {
    return String.format("[%d] %-20s | Cat: %-12s | Precio: S/.%7.2f | Stock:
    %4d",
        id, nombre,
        categoria != null ? categoria.getNombre() : "N/A",
        precio, stock);
}
}

```

//Clase Proveedor

```

package com.mycompany.caso3;
public class Proveedor {

    // Atributos privados (encapsulamiento)
    private int idProveedor;
    private String nombre;
    private String telefono;
    private String email;

    // Constructor
    public Proveedor(int idProveedor, String nombre, String telefono, String
    email) {
        this.idProveedor = idProveedor;
        this.nombre = nombre;
        this.telefono = telefono;
        this.email = email;
    }

    // Usando Getters y Setters
    public int getIdProveedor() {
        return idProveedor;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
}

```

```

    }

    public String getTelefono() {
        return telefono;
    }

    public void setTelefono(String telefono) {
        this.telefono = telefono;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        // Validación básica de email
        if (email != null && email.contains("@")) {
            this.email = email;
        } else {
            System.out.println("[ERROR] El email ingresado no es válido: " +
email);
        }
    }

    // Método de representación
    @Override
    public String toString() {
        return "Proveedor{id=" + idProveedor +
            ", nombre=" + nombre +
            ", telefono=" + telefono +
            ", email=" + email + "}";
    }
}

```

2.4. CASO 4

// Clase Docente que hereda de Persona

```
public class Docente extends Persona {
```

```
    private String materia;
```

```
    public Docente(String nombre, int edad, String materia) {
        super(nombre, edad); // Llamada al constructor de la clase Persona
        this.materia = materia;
    }

```

```
    public void presentar() {
```

```
        System.out.println("Hola, soy un docente de la materia de " + materia +
        ".");
    }

    // Métodos getter y setter
    public String getMateria() {
        return materia;
    }

    public void setMateria(String materia) {
        this.materia = materia;
    }
}

//CLASE ESTUDIANTE
public class Estudiante extends Persona{

    private String carrera;

    public Estudiante(String nombre, int edad, String carrera) {
        super(nombre, edad); // Llamada al constructor de la clase Persona
        this.carrera = carrera;
    }

    @Override
    public void presentar() {
        System.out.println("Hola, soy un estudiante de la carrera de " + carrera +
        ".");
    }

    // Métodos getter y setter
    public String getCarrera() {
        return carrera;
    }

    public void setCarrera(String carrera) {
        this.carrera = carrera;
    }
}

public class Persona {
    private String nombre;
    private int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }
}
```

```
}

public void presentar() {
    System.out.println("Hola, soy una persona.");
}

// Métodos getter y setter
public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public int getEdad() {
    return edad;
}

public void setEdad(int edad) {
    this.edad = edad;
}
}

// Clase Main para probar Herencia y Polimorfismo
public class Main {
    public static void main(String[] args) {
        // Crear objetos de tipo Estudiante y Docente
        Persona estudiante = new Estudiante("Carlos", 20, "Ingenieria");
        Persona docente = new Docente("María", 40, "Matematicas");

        // Llamar al método presentar, que se comporta de manera diferente
        // dependiendo del tipo de objeto (polimorfismo)
        estudiante.presentar(); // Llamada al método de Estudiante
        docente.presentar(); // Llamada al método de Docente
    }
}
```

Análisis:

La herencia permite reutilizar atributos y métodos comunes de Persona, como nombre, edad y presentación(). El polimorfismo permite usar una referencia de tipo Persona para ejecutar comportamientos diferentes según si el objeto real es Estudiante o Docente.

III. CONCLUSIONES

3. CONCLUSIONES POR CASO

3.1. CASO 1

El encapsulamiento protege los atributos usando private y permite acceso controlado mediante métodos get y set.

El constructor facilita la creación de objetos ya inicializados, haciendo el código más limpio y eficiente.

Estos conceptos ayudan a que el programa sea más seguro, ordenado y fácil de mantener.

3.2. CASO 2:

Este sistema bancario pone en práctica las bases de la programación Orientada a Objetos en Java de forma eficiente, por medio del cual da un correcto uso de atributos y métodos estáticos (gestión de información del sistema y el registro de cuentas generadas). Siendo la encapsulación lo que protege la información de entrada, mientras la abstracción logra un funcionamiento interno al sistema bancario sin olvidar que la herencia y polimorfismo ofrecen un manejo sólido de errores particulares para contextos del ámbito bancario.

3.3. CASO 3

El análisis previo mediante requerimientos, historias de usuario y diagramas UML constituye una etapa fundamental en el desarrollo de software orientado a objetos. Definir correctamente los requerimientos funcionales, junto con las historias de usuario, garantizan que el sistema responda a las necesidades reales del usuario. A su vez, el diagrama de clases UML permite visualizar la estructura del sistema y las relaciones entre clases antes de iniciar la programación, sentando las bases para un desarrollo organizado.

3.4. CASO 4

La herencia y el polimorfismo permiten crear programas más organizados, reutilizables y fáciles de mantener. En este caso, la clase Persona contiene características comunes, mientras que Estudiante y Docente reutilizan esa información y adaptan su comportamiento según su rol. Esto evita repetir código y facilita agregar nuevas clases en el futuro.

REFERENCIAS

Deitel, P. J. (2012). Java: Como Programar.

Java. (s.f.). Java. Obtenido de Java: <https://www.java.com/es/download/help/whatisjava.html>

w3school. (s.f.). w3school. Obtenido de w3school:

https://www.w3schools.com/java/java_user_input.asp

Webinars, O. (s.f.). OpenWebinars. Obtenido de OpenWebinars: <https://openwebinars.net/blog/introduccion-a-poo-en-java-excepciones/>